27/8/26 DAY 4:

Pointer Arithmetic

- Pointer arithmetic is the process of performing arithmetic operations on pointers.
- The operations include addition, subtraction, increment, and decrement.
- Pointer arithmetic is used to navigate through arrays and data structures in memory.

Pointers and Arrays

- Pointers and arrays are closely related in C and C++.
- An array name acts as a pointer to the first element of the array.
- You can use pointer arithmetic to access array elements.

Arrays using pointers

- You can create arrays using pointers by dynamically allocating memory for the array elements.
- Example:

```
int* arr = new int[size]; // dynamically allocate an array of integers
```

Pointer vs Reference

- A pointer is a variable that stores the memory address of another variable.
- A reference is an alias for an existing variable.

```
int a = 10;
int* ptr = &a; // pointer to a
int& ref = a; // reference to a
```

Pointer & Reference Comparison

Feature	Pointer	Reference
Syntax	`int* ptr = &a;`	`int& ref = a;`
Dereferencing	`*ptr`	`ref`
Memory occupation	Occupies memory for the pointer itself	Does not occupy additional memory
Memory Address	Can be changed to point to another variable	Cannot be changed to refer to another variable
Initialization	Must be initialized with an address	Must be initialized with an existing variable

Feature	Pointer	Reference
Reassignment	Can be reassigned to point to a different variable	Cannot be reassigned to refer to a different variable
Nullability	Can be null	Cannot be null
Usage	Used for dynamic memory management and data structures	Used for function parameters and operator overloading

Passing pointers to functions

We can pass pointers to functions to allow the function to modify the original variable's value or to work with large data structures efficiently.

```
void swap(int* a, int* b) { // pointer to int parameters
    int temp = *a;  // dereference to get the value
    *a = *b;
    *b = temp;
}
int main() {
    int x = 5, y = 10;
    swap(&x, &y);   // pass addresses of x and y
    // Now x is 10 and y is 5
}
```

const and Pointer combinations

const

- The `const` keyword can be used with pointers to indicate that the value being pointed to is constant.
- Once the variable is declared as `const`, its value cannot be changed or reassigned.

```
const int a = 10; // a is a constant integer

a = 20; // Error: cannot modify a constant variable
```

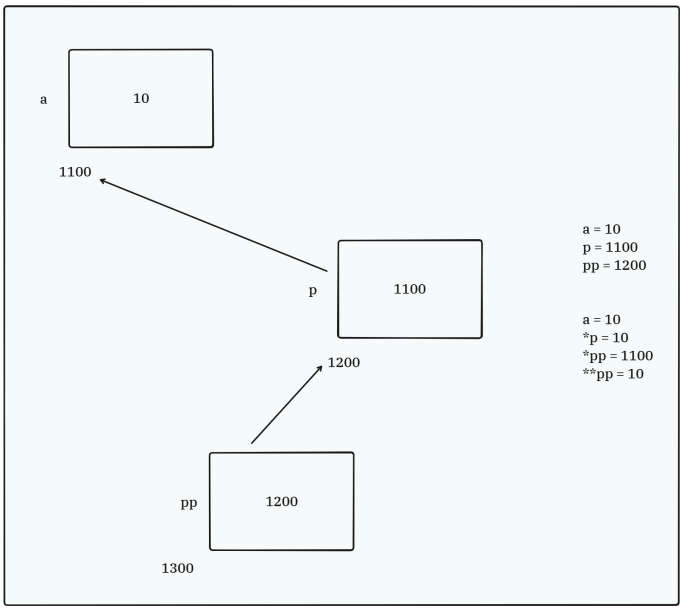
const pointer combinations

- const int* ptr & int const *ptr : pointer to a constant integer (value cannot be changed)
- int* const ptr : constant pointer to an integer (address cannot be changed)
- const int* const ptr : constant pointer to a constant integer (neither the value nor the address can be changed)

Pointer to Pointer

- A pointer to a pointer is a variable that stores the address of another pointer. It allows for multiple levels of indirection.

```
int a = 10;
int* ptr = &a; // pointer to int
int** ptr2 = &ptr; // pointer to pointer to int
```



Limitations of Procedural Programming

- Procedural programming is a programming paradigm that is based on the concept of procedures or functions.
- It is a step-by-step approach to solving problems, where the program is divided into smaller functions that perform specific tasks.
- It does not provide a clear way to model real-world entities and their interactions.
- It can lead to code that is difficult to maintain and extend, especially as the program grows in size and complexity.
- It consists of functions and data, but the data is not organized, leading to potential security risks due to data manipulation. Functions and data are mixed together and exist separately, making it complex to handle as the application grows.

Example of Procedural Programming

```
// student entity
student_rollno, student_name;

getStudentDetails(rollno, name){

}

setStundentDetails(rollno name){

}

// course entity
course_id, course_name;
```

```
getCourseDetails(id, name){

}

setCourseDetails(id, name){

}

// Both student and course entities are present in the same scope, but they are
not related to each other.
```

Limitations of C Structures

- C structures are used to group related data together, but they have limitations in terms of functionality and organization.
- C structures cannot contain member functions, which means they cannot encapsulate behavior along with data. This makes it difficult to model real-world entities that have both state and behavior.

```
struct Student{
    // state: data member
    int rollNo;
    char* name[100];
    double marks;

    // behaviour: member function cannot be included in C structures
};
```

Real-World Modeling Concept

- Real world entities have state and behaviour.
- State: Attributes of the entity, which can be represented as data members in a class.
- Behaviour: Functionalities of the entity, which can be represented as member functions in a class.
- Identity: Each entity has a unique identity, which can be represented by the memory address of the object in programming.

Introduction to Object-Oriented Programming

OOP (Object-Oriented Programming) is a programming paradigm that uses objects and classes to model real-world entities and their interactions. It provides a way to organize code in a more modular and reusable manner, making it easier to manage complex software systems.

Classes and Objects

Class

- class is a user-defined data type that serves as a blueprint for creating objects.
- It encapsulates data for the object and methods to manipulate that data.

- A class is a template for creating objects.
- A class defines the properties and behaviors that an object of that class will have.

Object

- An object is an instance of a class.
- It is a concrete entity that has state and behavior defined by its class.
- Each object has its own memory location and can be uniquely identified.

Class Definition Syntax

```
class ClassName{
    // data members
    // member functions
}; // class ends here..
```

```
class Student{
    //state: data member
    // behaviour: member function
};  // class ends here..
Student s // will have state & behaviour
```

Data Members

- Data members are the attributes of the object that hold the state of the object.
- They are variables defined inside a class that represent the properties of the object.

Example:

```
rollno, name, age, marks, course, address, bg, dob, grade, gender, mobno.
```

Member Functions

- Member functions are the methods defined inside a class that represent the behavior of the object.

Example:

```
showDetails(), showResult(), updateDetails(), acceptDetails()
```

Object Creation

- The syntax for creating an object of a class is as follows:

```
ClassName object_name(variable_name)
Student s; // example
```

Access Specifiers

- Access specifiers are keywords used to define the accessibility of class members (data members and member functions) from outside the class.
- They determine the level of access control for the members of a class.

private

- Members declared as private are accessible only within the class itself.
- They cannot be accessed directly from outside the class or by derived classes.

public

- Members declared as public are accessible from anywhere in the program.
- They can be accessed directly from outside the class and by derived classes.

protected

- Members declared as protected are accessible within the class itself and by derived classes.
- They cannot be accessed directly from outside the class.

Comparison of Access Specifiers:

Access Specifier	Accessibility
private	Accessible only within the class
public	Accessible from anywhere in the program
protected	Accessible within the class and by derived classes

Abstraction Concept

- Abstraction is the process of hiding the implementation details and showing only the essential features of an object.
- It allows the programmer to focus on what an object does instead of how it does it.
- It is achieved in C++ through the use of classes and objects, where the internal workings of a class are hidden from the outside world, and only the necessary interfaces are exposed.
- Access specifiers (private, public, protected) play a crucial role in achieving abstraction by controlling the visibility of class members.

Encapsulation Concept

- Encapsulation is the process of bundling data members and member functions that operate on the data into a single unit called a class.

- It restricts direct access to some of an object's components, which is a way of preventing unintended interference and misuse of the data.
- It helps in achieving data hiding and improving the security of the data.

> When the class is created encapsulation is achieved, and when the object is created abstraction is achieved.

Getters & Setters

- Getters and setters are member functions that provide controlled access to the private data members of a class.
- A getter function is used to retrieve the value of a private data member.
- A setter function is used to set or update the value of a private data member.
- A setter function can include validation logic to ensure that the data being set is valid, while a getter function can provide read-only access to the data. Example:

```
class Student {
private:
    int rollNo; // private data member
public:
    // Getter function
    int getRollNo() const {
        return rollNo;
    }
    // Setter function
    void setRollNo(const int r) {
        rollNo = r;
    }
};
```

Basic Object Memory Concept

- When an object is created, memory is allocated for its data members.
- The memory for an object can be allocated on the stack or on the heap, depending on how the object is created.

Padding Concept

- Padding is the process of adding extra bytes to a data structure to ensure that its members are aligned in memory according to the architecture's requirements.
- It helps in improving the performance of memory access by ensuring that data members are aligned to their natural boundaries.

Objects on Stack and Heap

- Objects can be created on the stack or on the heap, depending on how they are declared.
- Stack allocation is faster and automatically managed, while heap allocation provides more flexibility but requires manual memory management.

- Objects created on the stack are automatically destroyed when they go out of scope, while objects created on the heap must be explicitly deleted to free the memory.

Tomorrow's Topics:

Arrays of Objects

Function Overloading & Default Arguments

Constructor Concept

Default Constructor

Parameterized Constructor

Multiple Constructors

Constructor Overloading

Dynamic Initialization of Objects

Constructor Initializer List

Destructor

Object Lifecycle

Namespaces

std Namespace